ASSIGNMENT REPORT

CO3 – CRYPTOGRAPHY AND NETWORK SECURITY

Evaluate Cryptographic Hash Algorithms, Digital Signatures, Kerberos and X.509 Authentication Frameworks

STUDENT DETAILS

Name: Samarapaku Madhuri

Registration Number: 192465036

Programme: B.E. – CSE (Cybersecurity)

Institution: Saveetha School of Engineering

Course Outcome: CO3

Total Marks: 50

Submitted as an academic assignment report

1. ASSIGNMENT QUESTIONS

1. Q1 – What Makes a Hash Function Trustworthy? (6 Marks)
2. Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)
3. Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)
4. Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)
5. Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)
6. Q6 – Advanced Security Mini-Investigations (7 Marks)
7. Q7 – Design a Secure Authentication and Integrity Scheme (10 Marks)

2. ABSTRACT

This report examines cryptographic hash functions, message authentication codes, HMAC, digital signatures, X.509 certificate validation, Kerberos authentication, and the design of a secure authentication and integrity scheme for medical-device firmware updates. It connects mathematical properties with practical attack scenarios, implementation choices, trust models and deployment risks.

Toy parameters and educational implementations are used only to clarify principles. They are not production cryptography. No unmeasured experimental timing, avalanche average or collision count is claimed as an executed result; all experimental result fields marked “to be measured” must be completed by the students after running the programs.

3. CONTENTS

- Q1 – Hash-function properties, avalanche behaviour and collision effort
- Q2 – Hashes, MACs, HMAC and length-extension resistance
- Q3 – Digital signatures and implementation failures
- Q4 – X.509 certificate chains and validation
- Q5 – Kerberos protocol analysis
- Q6 – Advanced signature investigations and framework comparison
- Q7 – Secure medical-device firmware-update design
- Overall Conclusion
- References

Q1 – WHAT MAKES A HASH FUNCTION TRUSTWORTHY? (6 MARKS)

1.1 Security properties as attack scenarios

Pre-image resistance means that, given a digest y, an attacker should not feasibly find any message m such that H(m)=y. It protects a hidden message or secret-derived value from being recovered from its digest.

Second-pre-image resistance means that, given a particular message m, an attacker should not feasibly find a different message m′ with H(m′)=H(m). It prevents substitution of a chosen alternative for an already established message.

Collision resistance means that it should be infeasible to find any two distinct messages m and m′ with the same digest. The attacker chooses both messages, so collision attacks are generally easier than second-pre-image attacks.

Avalanche behaviour requires a small input change, ideally one bit, to change approximately half of the output bits in an unpredictable way. It prevents an attacker from tracking a simple relationship between input and digest.

1.2 Ideal avalanche result and weaknesses of toy hashes

For an ideal n-bit digest, each output bit changes with probability 1/2 when one input bit is flipped. If X is the number of changed output bits, then X follows a binomial distribution: X ~ Binomial(n, 1/2). Therefore E[X]=n/2 and Var(X)=n/4. For SHA-256, the ideal expected number is 128 changed bits, although an individual trial may be above or below 128.

```
def byte_sum(data): return sum(data) % 256

def xor_fold(data):
    value = 0
    for b in data: value ^= b
    return value
```

These functions have tiny output spaces and preserve algebraic structure. A byte-sum changes predictably when a byte is altered, while XOR permits cancellation: changing two equal bytes leaves the result unchanged. Many different messages necessarily share each value, so pre-images and collisions can be constructed cheaply. Neither function provides an avalanche effect or meaningful second-pre-image resistance.

A fixed-length digest must have collisions by the pigeonhole principle. The set of possible messages is effectively unbounded, while an n-bit digest has only 2^n possible values. Consequently, distinct messages must map to the same value even when finding such a pair is computationally difficult.

1.3 Avalanche experiment

The experiment should use short, medium and long messages, for example 8, 64 and 1024 bytes. For each trial, select a message, flip exactly one input bit, hash both messages and count the one-bits in digest1 XOR digest2. At least 30 repeatable trials per size provide a useful educational sample; the random seed must be recorded.

```
import hashlib, random

def changed_bits(a, b): return sum((x ^ y).bit_count() for x, y in zip(a, b))

def trial(size, seed=192465036, repeats=30):
    rng = random.Random(seed + size)
```

```
    values = []
    for _ in range(repeats):
        m = bytearray(rng.randbytes(size)); i = rng.randrange(size * 8)
        altered = bytearray(m); altered[i // 8] ^= 1 << (i % 8)
        a = hashlib.sha256(m).digest(); b = hashlib.sha256(altered).digest()
        values.append(changed_bits(a, b))
    return sum(values) / len(values)
```

The returned averages for 8, 64 and 1024 bytes are to be measured. No numerical average is claimed here. Small deviations from 128 are expected because the sample is finite; a systematic departure would suggest a flawed implementation or a non-ideal construction.

1.4 Birthday-bound collision search

For k-bit truncation, store each truncated digest and stop when a repeated value is found. The following code searches k = 16, 20, 24, 28 and 32 without claiming that it has been run.

```
def truncated_sha256(data, k):
    return int.from_bytes(hashlib.sha256(data).digest(), "big") >> (256-k)

def find_collision(k):
    seen = {}; i = 0
    while True:
        m = f"message-{i}".encode(); d = truncated_sha256(m, k)
        if d in seen: return i + 1, seen[d], m
        seen[d] = m; i += 1
```

k	Theoretical scale 2^(k/2)	Observed hashes, trial 1	Observed hashes, trial 2
16	256	To be measured	To be measured
20	1,024	To be measured	To be measured
24	4,096	To be measured	To be measured
28	16,384	To be measured	To be measured
32	65,536	To be measured	To be measured

This is also the graph-ready results table: plot k on the horizontal axis and measured hashes on a logarithmic vertical axis, with the theoretical 2^(k/2) curve as a comparison. The birthday approximation follows P(collision by q) ≈ 1 − exp(−q(q−1)/(2·2^k)); probability becomes substantial when q is on the order of 2^(k/2), not 2^k.

Finding any collision allows the attacker to choose both messages. A target or second-pre-image attack fixes one message in advance and is therefore expected to require approximately 2^k work for an ideal n-bit hash. Digest length must account for collision security, pre-image security, future computation and the value of the signed

data. MD5 and SHA-1 are unsuitable for new security designs because practical collision attacks have undermined their collision-resistance assumptions. A modern policy should use SHA-256 or stronger and approved signature schemes.

Q2 – INTEGRITY IS NOT THE SAME AS AUTHENTICATION (7 MARKS)

2.1 Hashes, MACs and the threat model

A public SHA-256 digest on the same webpage supplies an integrity comparison only if the digest itself is obtained through a trusted channel. An attacker who can replace both the file and its webpage digest can recompute the digest. A hash has no secret and therefore cannot authenticate its publisher.

For an educational fixed-block construction, pad the message with a one bit, zero bits and an encoded original length so that it ends on a block boundary. Process each block with a chaining function and publish the final state. The naive MAC is H(K || M). HMAC is H((K′ XOR opad) || H((K′ XOR ipad) || M)), where K′ is the block-sized key.

```
def educational_mac(key, message):
    return toy_hash(key + message)
def educational_hmac(key, message):
    k = normalize_to_block(key); ipad = bytes([0x36])*BLOCK; opad = bytes([0x5c])*BLOCK
    return toy_hash((k ^ opad) + toy_hash((k ^ ipad) + message))
```

A length-extension demonstration exposes a structural weakness of Merkle–Damgård hashing. If an attacker knows H(K || M), the digest can act as the chaining state for M followed by its padding and an appended suffix. The attacker needs M's length, the public construction and the observed tag; the attacker does not need K. The attack is conceptual here and must be implemented only with the educational toy construction.

HMAC avoids exposing a usable state for K || M. The inner hash is nested inside a separately keyed outer hash, and the two distinct pads provide domain separation. HMAC therefore resists the ordinary length-extension attack when correctly implemented, although key compromise, weak keys, side channels and implementation errors remain risks.

Mechanism	Integrity	Authentication	Non-repudiation
Hash only	Detects accidental change when reference digest is trusted	No	No
Naive H(K || M)	Yes against outsiders, subject to construction weakness	Yes for holders of K	No; both parties can generate tags
HMAC	Yes, with a strong hash and secret key	Yes for holders of K	No; shared-key evidence is not unique

For download verification, the organisation should use a digital signature anchored in a certificate or another authenticated public-key channel, not a bare public hash. For internal message protection, HMAC is appropriate when the communicating parties can protect a shared secret. Residual risks include stolen keys, poor key rotation, replay, compromised endpoints, untrusted distribution channels and incorrect implementation. None of the toy constructions should be deployed.

Q3 – DIGITAL SIGNATURES: HASH-THEN-SIGN (7 MARKS)

3.1 Hashing, signing and verification

Hash-then-sign compresses an arbitrary message into a fixed-size digest before applying the signature primitive. This improves efficiency and binds the signature to the complete message. Signing a raw long message with textbook RSA would expose algebraic structure, impose size restrictions and permit multiplicative manipulation; a standardized encoding and approved signature scheme are required.

```
def rsa_sign(m, d, n): return pow(int.from_bytes(sha256(m), "big"), d, n)

def rsa_verify(m, s, e, n): return pow(s, e, n) == int.from_bytes(sha256(m), "big")
```

For textbook RSA, let h=H(m), s=h^d mod n and ed ≡ 1 mod φ(n). Then s^e ≡ (h^d)^e ≡ h^(de) ≡ h mod n, under the RSA assumptions. Real RSA signatures must use a standardized encoding such as RSA-PSS; the displayed relation is an educational proof, not a deployment prescription.

For DSA, with private x, public y=g^x, nonce k and z=H(m), r=(g^k mod p) mod q and s=k⁻¹(z+xr) mod q. Verification computes w=s⁻¹, u1=zw and u2=rw, then v=((g^u1 y^u2 mod p) mod q); it succeeds because g^z·(g^x)^r raised to k/(z+xr) recovers g^k. ECDSA has the analogous elliptic-curve relation: R=u1G+u2Q and v=x-coordinate(R) mod n.

The hash supplies message binding and integrity; the private key supplies proof of control of the signing key; public-key validation supplies authentication only when the public key is correctly bound to an identity. Non-repudiation additionally requires exclusive key custody, reliable identity proof, trustworthy records and a legal framework.

Test	Expected result	Reason
Original message and signature	Accept	Digest and public-key equation agree
Modified message	Reject	Digest changes
Modified signature	Reject	Public-key verification fails
Correct signature with wrong public key	Reject	The verification key does not correspond to the signer

The test table records expected outcomes; actual execution evidence must include the program output. If the same nonce k is reused for two DSA/ECDSA signatures, $s1=k^{-1}(z1+xr)$ and $s2=k^{-1}(z2+xr)$. Thus $k=(z1-z2)/(s1-s2) \bmod q$ and $x=(s1k-z1)/r \bmod q$. This is catastrophic because the private key can be recovered.

Textbook no-hash RSA is multiplicative: if $s1^e=m1$ and $s2^e=m2$, then $(s1s2)^e \equiv m1m2 \bmod n$. An attacker can therefore combine signatures on selected messages to create a signature on their product. Proper hashing, encoding and modern padding remove this simple algebraic forgery.

A valid signature supports non-repudiation only when the private key was under the signer's exclusive control, the certificate identity was reliably verified, the signature time can be established, audit evidence is protected and the algorithm was valid at signing time. The signer may dispute custody, claim compromise before signing, challenge certificate status, or question the timestamp. A trusted certificate authority and timestamp authority can strengthen evidence, but neither eliminates operational or legal uncertainty.

Q4 – TRUST THROUGH X.509 CHAIN VALIDATION (7 MARKS)

An X.509 v3 certificate contains a subject, issuer, serial number, validity interval, subject public key information, extensions and a CA signature over the encoded certificate body. A conceptual hierarchy is Root CA → Intermediate CA → End-Entity. The root is self-signed and installed as a trust anchor; the intermediate is authorised by the root; the end-entity certificate identifies a server, device or person.

```
validate(chain, hostname, now, trust_store):
  require chain ends at a configured trust anchor
  for each child, issuer in chain:
    require child.issuer == issuer.subject
    require verify(issuer.public_key, child.signature, child.tbs)
    require issuer.valid_from <= now <= issuer.valid_to
    require issuer.basic_constraints.CA == true and key_usage permits signing
  require leaf.valid_from <= now <= leaf.valid_to
  require hostname matches subjectAltName
  require revocation_status(leaf and relevant issuers) == good
  return trusted
```

Signature checks prevent undetected certificate alteration. Issuer–subject matching prevents an unrelated certificate from being inserted. Validity checks prevent use outside the authorised time. Basic constraints and key usage stop an end entity from issuing certificates. Name matching prevents a valid certificate for one host being used for another. Revocation checks address known compromise or mis-issuance. Trust-anchor validation limits the chain to an administrator-approved root.

Failure case	**Rejecting step**
Expired certificate	Validity-period check
Not-yet-valid certificate	Validity-period check
Wrong issuer	Issuer–subject matching or signature check
Tampered certificate field	Certificate signature check
Hostname mismatch	Subject alternative name check
End entity used as CA	Basic constraints and key usage
Revoked certificate	CRL or OCSP revocation check

CRLs are signed, periodically published lists and can be large or stale between updates. OCSP returns a smaller, more current status response but may reveal which certificate a client is using and depends on responder availability. A fail-open policy accepts a certificate when revocation cannot be checked; it improves availability but permits use of a revoked certificate. A fail-closed policy protects security more strongly but can deny service during an outage.

The root trust anchor is an administrative assumption: cryptography can verify that the root signed an intermediate, but cannot prove that the root was selected honestly or remains operationally trustworthy. X.509 makes a signature meaningful by binding a verified identity statement to a public key. If a CA is compromised or mis-issues a certificate, short certificate lifetimes, constrained intermediates, monitoring, rapid revocation, key separation, transparency records and emergency trust-store updates limit the duration and scope of harm.

Q5 – KERBEROS: AUTHENTICATION WITHOUT SENDING PASSWORDS (6 MARKS)

Kerberos uses a trusted Key Distribution Center consisting of an Authentication Server (AS) and Ticket-Granting Server (TGS). The client proves knowledge of a long-term key derived from its password without transmitting the password. The KDC issues short-lived tickets and session keys.

1. C → AS: IDc, IDtgs, nonce
2. AS → C: {Kc,tgs, lifetime, nonce}Kc, TGT={IDc, address, lifetime, Kc,tgs}Ktgs
3. C → TGS: TGT, {IDc, timestamp}Kc,tgs, IDs

4. TGS → C: {Kc,s, lifetime}Kc,tgs, ST={IDc, lifetime, Kc,s}Ks

5. C → S: ST, {IDc, timestamp}Kc,s

6. S → C: {timestamp + 1}Kc,s

The TGT allows the client to request service tickets without repeatedly exposing the password-derived key. The user long-term key protects the AS reply only; subsequent communication uses session keys. The authenticator proves freshness because it contains a timestamp encrypted under the session key. The service validates the ticket with its long-term service key, checks the authenticator and returns a response encrypted with the session key, providing mutual authentication.

A small educational simulation should model principals, long-term symmetric keys, a clock, ticket lifetime and a replay cache. Encryption may be represented by authenticated encryption in the implementation; plaintext substitution must not be described as secure encryption. The service accepts only if the ticket decrypts, the client identity matches, the timestamp is within the permitted skew, the ticket is unexpired and the authenticator has not appeared previously.

Test	Expected result	Reason
Fresh ticket and authenticator	Accept	Valid keys, lifetime and freshness
Replay without replay cache	Accept, demonstrating weakness	Timestamp alone does not remember prior use
Replay with replay cache	Reject	Authenticator identifier is already recorded
Expired ticket	Reject	Ticket lifetime has ended
Clock skew beyond policy	Reject	Timestamp is outside the permitted window
KDC unavailable after cached ticket expires	Reject new service request	No trusted issuer is available for a fresh ticket

A stolen service ticket may permit impersonation until expiration if the attacker can also construct or replay acceptable authenticators; short lifetimes, channel binding and replay caches reduce the risk. A captured AS reply can support offline password guessing if the password-derived key is weak, so strong passwords, pre-authentication and rate limiting are important. The password is never sent because the AS decrypts or verifies a response protected by a password-derived key. The service learns the client identity asserted in the ticket, ticket validity and session context, but not the password.

Evidence for replay prevention consists of one accepted fresh request followed by a rejected duplicate with the same authenticator, together with a log showing the timestamp, replay-cache decision and reason. A separate run without the cache should show why the cache is necessary.

Q6 – ADVANCED SECURITY MINI-INVESTIGATIONS (7 MARKS)

A collision attack finds any two messages with the same digest, whereas a second-pre-image attack starts from a specified message and seeks a different message with the same digest. With a truncated hash, two prepared messages M1 and M2 can satisfy Hk(M1)=Hk(M2). A signature over the truncated digest of M1 then verifies for M2. Both messages must be prepared before signing because the attacker needs the collision pair and cannot generally adapt M2 after observing an arbitrary signed message.

The historical MD5 and SHA-1 collision lessons show that certificate signatures are only as strong as the digest used in the signed data. A CA should prohibit obsolete collision-broken hashes, select SHA-256 or stronger, follow approved signature standards and review security strength against the expected certificate lifetime.

Scheme	Key material	Randomness	Verification and assumptions
RSA-PSS	Modulus, private exponent, public exponent	Salt and secure encoding randomness	Public exponent check; security requires correct padding and key protection
DSA/DSS	Finite-field public and private values	Unique secret nonce	Modular verification; nonce secrecy and uniqueness are essential
ECDSA	Elliptic-curve point and scalar	Unique secret nonce or approved deterministic method	Curve-point verification; private-key custody supports attribution
ElGamal	Group generator, public value and private exponent	Fresh invertible signing nonce	Equation g^m = y^r r^s; randomness failure exposes the key
Schnorr	Group public point and private scalar	Fresh commitment nonce and unpredictable challenge	Linear verification; soundness depends on challenge binding

In the Schnorr protocol, the signer chooses r and publishes R=g^r, receives challenge c and returns z=r+cx. The verifier checks g^z=R·Y^c, where Y=g^x. If c is predictable, a forgery can be prepared for a chosen challenge. Reusing r in two transcripts gives z1−z2=(c1−c2)x and reveals x.

For ElGamal, choose a group of order q, generator g, private x and public y=g^x. Select a fresh k with gcd(k,q)=1, set r=g^k and s=k⁻¹(m−xr) mod q. Verification checks g^m ≡ y^r r^s mod p because y^r r^s=g^{xr}g^{k(m−xr)}=g^m in the appropriate modular formulation. The following educational test cases must be run with toy parameters: valid message/signature — accept; altered message — reject; altered r or s — reject. The resulting accept/reject values are to be measured, not assumed to be production evidence.

Framework	Key type	Trusted third party	Typical environment
Kerberos	Shared symmetric keys and tickets	KDC	Managed campus or enterprise SSO
X.509/PKI	Public/private key pairs and certificates	CA and trust anchors	Internet TLS, devices and signed contracts

Kerberos depends on an available and trusted KDC, while X.509 depends on trusted roots, certificate status infrastructure and private-key protection. A Kerberos ticket authenticates a user to a service but does not provide unique evidence to a later third party because the service and KDC context use shared secrets. A private-key signature can support non-repudiation when identity, custody, time and legal evidence are sound. Pairwise symmetric keys require $n(n-1)/2$ relationships; a KDC reduces long-term relationships to roughly n client-to-KDC keys plus service keys; PKI gives each participant a key pair and certificate, with trust-management rather than pairwise secret distribution. Kerberos is appropriate for campus SSO; X.509-based signatures are more appropriate for legally significant contracts, subject to jurisdiction, qualified trust services and evidence requirements.

Q7 – DESIGN A SECURE AUTHENTICATION AND INTEGRITY SCHEME (10 MARKS)

7.1 Problem and threat model

The system distributes firmware to infusion pumps that may be offline and may have unreliable clocks. An attacker may intercept, replay, modify or suppress updates, substitute an older vulnerable image, compromise a hospital network, steal a pump, or attempt to use a revoked certificate. The manufacturer signing key, certificate authority and update service are high-value assets. The design must not depend on Kerberos for offline firmware authenticity.

7.2 Student A – independent initial proposal

- Goals: use SHA-256 for integrity, a manufacturer signature for origin and non-repudiation, certificates for identity, and a version counter for rollback protection.
- Primitives: use an approved modern signature, authenticated encryption for confidential payloads when required, and a separate MAC only for an authenticated online service session.
- Key material: the manufacturer protects a signing key in an HSM; each pump stores the manufacturer verification key or trust anchor; the hospital server has its own certified key.
- Flow: sign a manifest containing device family, version, hash, size, policy and expiry; distribute the image and manifest; verify the certificate, signature, digest and monotonic version before installation.

7.3 Student B – independent initial proposal

- Goals: make updates authentic and recoverable during offline operation while retaining a local audit record.
- Primitives: use a signed manifest, SHA-256 digest, device-unique authenticated encryption for downloaded packages and optional Kerberos authentication only when a pump contacts the hospital server.
- Key material: store a manufacturer root key, a rotating update-signing key, device identity credentials and a recovery key in protected hardware.
- Flow: request metadata when connected, validate the signed manifest offline, reject versions below the stored minimum, install atomically and retain the previous known-good image for recovery.

7.4 Design-comparison record

Topic	Agreement	Difference and decision
Integrity	Both select SHA-256 or stronger	Retain digest in a signed manifest, not as a public standalone value
Authentication	Both select public-key verification	Use certificate chain validation; keep Kerberos only for online service sessions
Availability	Both support offline verification	Use a cached trust anchor and a monotonic version counter
Threat coverage	Both identify replay and stolen keys	Add rollback, wrong clock, revocation, recovery and compromised-root procedures

The proposals agree on signed metadata and offline verification. Student A emphasises a minimal signed manifest, while Student B adds package confidentiality, recovery and online service authentication. The final design retains the signed manifest, adds authenticated encryption where confidentiality is justified, and explicitly treats clock-independent versioning and key recovery as safety requirements.

7.5 Joint final design

The manufacturer maintains a root of trust and separate online and offline signing roles. A release manifest contains a canonical format version, device family, firmware version, minimum accepted version, image length, SHA-256 digest, algorithm identifiers, release identifier, optional expiry and the manufacturer's signature. The image may be encrypted with authenticated encryption when confidentiality is needed, but encryption does not replace the signature.

- Issuance: release approval requires two-person control; the signing key remains in an HSM; certificates identify authorised signing roles.
- Storage: pumps store verification keys and certificate constraints in protected storage; private keys never leave their authorised devices.
- Rotation and revocation: use overlapping signing certificates, short validity where connectivity permits, emergency trust-list updates and a recovery signing key held separately.

- Offline and clock failure: verify certificate policy using a trusted release record and monotonic version counter; do not rely on a pump wall clock alone. When online, refresh status and time information.
- Stolen key: stop the affected signer, publish a revocation or trust-list update, use the recovery key, investigate signed releases and require a secure boot or service process to install the new trust state.

```
sign(image, metadata, signing_key):
    manifest = canonicalize(metadata + SHA256(image) + length(image))
    certificate = authorised_signing_certificate()
    signature = approved_signature(signing_key, manifest)
    return package(image, manifest, signature, certificate)

verify(package, pump_state):
    validate_certificate_chain(package.certificate, trusted_roots, current_policy)
    require signature_valid(package.certificate.public_key, package.signature, package.manifest)
    require package.manifest.device_family == pump_state.family
    require package.manifest.version >= pump_state.minimum_version
    require SHA256(package.image) == package.manifest.image_hash
    install_atomically_and_record_audit_event()
```

Message flow: Manufacturer → signed manifest plus image → distribution service → pump; the pump validates certificate and signature, checks device family and version, hashes the image, optionally decrypts and authenticates it, installs atomically and records the release identifier. Pump ↔ hospital server authentication may use Kerberos when online, but the pump must verify firmware locally without a KDC.

7.6 Security-argument validation

Student A should argue that a generic collision against an n-bit digest costs approximately 2^(n/2), a second-pre-image attack approximately 2^n, and a signature forgery should be infeasible at the selected security level. Key compromise, root compromise, a malicious authorised release, parser bugs, insecure boot firmware, physical extraction and denial of service remain residual risks. These are arguments based on assumptions, not proof that a deployment is secure.

Attack or test	Expected result	Verification cost
Valid update	Accept	One image hash, one signature verification, chain checks
Modified image	Reject	One image hash plus signature/manifest checks
Modified signature	Reject	One signature verification
Replayed older version	Reject	Version comparison plus normal checks
Expired or revoked certificate	Reject when status information is available and policy requires it	Chain and status checks
Wrong signing key	Reject	Certificate-chain and signature checks

Student B must execute these cases and record measured hash operations, signature verifications, metadata size and elapsed time. All timing fields are to be measured; no invented values are reported.

Baseline comparison	Bare published SHA-256	Signed-manifest design
Modified image with unchanged reference	Reject if reference is trusted	Reject
Attacker replaces image and reference	Accept incorrectly	Reject without signing-key compromise
Rollback to older valid image	Cannot detect by digest alone	Reject below minimum version
Origin authentication	None	Certificate and signature
Measured time and metadata	To be measured	To be measured

In cross-defence, Student A should explain how measured tamper tests, rollback tests and certificate tests support the design. Student B should explain that SHA-256 supplies digest binding, the signature supplies origin evidence, certificates supply identity binding, authenticated encryption supplies confidentiality and channel integrity where required, and the version counter supplies rollback resistance. Integrity, authentication and non-repudiation are not interchangeable properties.

The design costs include signature verification, certificate storage, trust-list maintenance, secure key management, release governance and recovery procedures. It can still fail through key compromise, a compromised root, incorrect policy, vulnerable boot code, unsafe parsing, loss of status information or physical attack. HMAC may be preferable for a closed system where all verifiers are controlled and non-repudiation is unnecessary; Kerberos is useful for online hospital service sessions, not for offline firmware authenticity.